Using Wild Cards

- Sometimes you will need to execute a command on a group of file instead of a single file.
  - Examples:
    - You want to **delete all log files**.
    - You want to **list all image files.**
    - You want to **copy old files** (ending with **.old**) to a **different place.**
- The solution for that is to use **Wild Cards** (also called `Globbing`)
- Wild cards are patterns that work as placeholders in file names and directory names that are used to apply the command on a group of files/directories that share something in their name.

> 💡 Remember wild cards are used for file names and directory names ONLY... For normal text another patterns are used (**Regular Expressions**).

The '*' Wild Card

The "*" can replace **any set of characters** (**including none**) in the file /directory name.

```
$rm *.php [will delete .php, a.php, b.php, c.php]
$rm p* [hyms7 files bs msh directories]
$rm *.* htm*
$rm -r *.* [hyms7 directories]
```

The '?' Wild Card

The "**?**" wild card stands for **any single character**

```
# f1.php , f.php
$rm f?.php # only f1.php will be removed
```

[<chars>] and [!<chars>]

We can have more restriction than the use of "?" by specifying a limited set of options for the character

- "[ars]": Stands for a **single character** from the list `a,r,s`
- "[!ars]": Stands for any s**ingle character** except for the list `a,r,s`
- "[2-5]": Stands for a **single character** from the range `2 to 5`
- "[a-l]": Stands for any **single character** except for the list `a to l`
- "[1-37-9]": Stands for `1,2,3,7,8,9` [mmno3 t3ml space]
- "[a-chk]": Stands for `a,b,c,h,k`

```
abc.txt
bbc.txt
rbc.txt

$rm [ab]bc.txt
# bc.txt won't be removed !
```

Examples:

```
$rm -r ab[c-fh-j]
removes the files/folders named abc,abd,abe,abf,abh,abi,abj

#abcd.txt [won't be removed]

$ls results-[0-9][0-9].log
lists the files named results-00.log to results-99.log
```

[[:<class Name>:]]

Stands for a **single character** belonging to the specified class

Some of used classes:

- `[[:alnum:]]` Alpha Numeric characters (a-z, A-Z, 0-9)
- `[[:alpha:]]` Alphabets (a-z, A-Z)
- `[[:digit:]]` Digits (0-9)
- `[[:lower:]]` Lower case characters (a-z)
- `[[:upper:]]` Upper case character (A-Z)

Examples:

```
$cp results-[[:digit:]][[:digit:]]-[[:alpha:]].log ~/log
```

Use of curly brackets “{}”

Curly brackets are used to **group selections**

Examples:

```
# spaces are not allowed [f.{log, txt}]
$rm f.{log,txt} # Remove any file that is called f and its ex
tensions is either log, txt
$cp *{.pdf,.doc} ~/documtents/
```

Note:

This also works

```
$rm *.log *.txt
$cp *.pdf *.doc ~/documents/
```

- A nice tip,
    - If you’re going to remove a bunch of files/directories using wild cards, a good idea is to list them first with the same wildcard pattern to make sure you are not doing a mistake and removing the wrong files, then replace “ls” with an “rm”

- Or if not sure, use the `"-i"` to confirm each file delete before it is deleted

Escape Sequence "\"

- Some special letters has a meaning (such as `space`, `*`, `"`, `'`, `(` ,...)
- It is not recommended to use these letters in file/directory names
- But, if we have to, then there is a special way of dealing with them,
- If we need to delete a file named "my results.txt"

```
$rm my results.txt  # will be treated as two separate files
$rm my\ rsults.txt  # \ is used to escape the white space
```

This is called "`Escaping the space letter`" which means changing its default meaning from a separator letter into a **general letter inside** the filename.

Examples of Escape Sequence:

Real File Name	File Name representation
my?file.log	my\?file.log
my*file.log	my*file.log
my file (today).txt	my\ file\ \(today\).txt
"my file"	\"my\ file\"
abc[!2]	abc\[!2\]



tab to autocomplete the command

Examples

```
Remove files that are called test-1 to test-99 using two ways

$rm test-??; $rm test-?
$rm test-*
$rm test[1-99]
```

- Can I copy a directory into itself?
  - Struct inside itself?
  - Recursion !!

How to create a file called “my results.txt”

```
$touch my\ results.txt

# What will be the output of this command
$touch my\?\ \* [my? *]
```